



ScholarRAG

An AI-Powered Academic Research Assistant
Retrieval-Augmented Generation over 22,000+ Research Papers,
Containerized and Orchestrated on Kubernetes

Domain of the Project

Cloud Computing, DevOps & Generative AI (RAG)

Team Mentor

Mr. Ujjwal Pratap Singh — Founder, Skill Nebula
linkedin.com/in/ujjwal-pratap-singh

Team Members

Ms. Akriti Yadav (Team Leader) · Mr. Anurag Singh · Ms. Shruti Gupta
Mr. Saurabh Chandra Shukla · Mr. Oum Gupta

Period of the Project

13 June 2026 to 28 June 2026



Declaration

The project titled “ScholarRAG – An AI-Powered Academic Research Assistant” was built as part of a Cloud Computing internship mentored by Mr. Ujjwal Pratap Singh, Founder of Skill Nebula. During the internship he trained the team in modern cloud technologies — AWS, Docker and Kubernetes — and set a 15-day challenge (13 June 2026 to 28 June 2026) to design, containerize and deploy this application end-to-end. I declare that to the best of my knowledge the members of the team mentioned below have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Ms. Akriti Yadav (Team Leader)

Mr. Anurag Singh

Ms. Shruti Gupta

Mr. Saurabh Chandra Shukla

Mr. Oum Gupta

Mentor's Name:

Mr. Ujjwal Pratap Singh

Designation – Founder, Skill Nebula

Mr. Ujjwal Pratap Singh

Founder, Skill Nebula

Mentor & Program Lead



Table of Contents

1. Executive Summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry Relevance of the Project
6. Learning & Reflection
7. Future Scope & Conclusion
8. Appendix – Deployment Commands & Code Snippets



Executive Summary

Objective

Build ScholarRAG, an AI-powered research assistant that lets users ask natural-language questions and upload research PDFs, then returns grounded, citation-backed answers drawn from a corpus of 22,000+ academic papers. The complete stack is containerized with Docker and deployed on a multi-node Kubernetes (kind) cluster fronted by an NGINX Ingress.

Methods

A FastAPI backend implements a Retrieval-Augmented Generation (RAG) pipeline: HyDE query expansion → bi-encoder semantic search over a Qdrant vector database → cross-encoder reranking → answer generation with Anthropic Claude, streamed to the browser over Server-Sent Events (SSE). A React single-page frontend, a SQLite user/chat store, Docker images per service, and Kubernetes manifests (Deployments, Services, Ingress) complete the 3-tier system.

Key Findings

- The full RAG pipeline works end-to-end: HyDE expansion, embedding with **all-MiniLM-L6-v2**, vector retrieval from the **papers** collection in Qdrant, and cross-encoder reranking with **ms-marco-MiniLM-L-6-v2** before Claude generates the final answer.
- The API surface is clean and REST-based under FastAPI: **/register**, **/login**, **/query**, streaming **/query/stream** (SSE), chat management (**/chats**, pin, star, delete, clear) and a **/health** probe.
- A tunable relevance floor (**RERANK_FLOOR**, env-configurable) filters weak matches, with a separate lower floor for uploaded-PDF queries so on-topic papers are never dropped.
- Kubernetes manifests run the backend (2 replicas, port 8000), frontend (2 replicas, port 80) and Qdrant (port 6333); the NGINX Ingress routes **/api/*** to the backend and everything else to the frontend, with proxy buffering disabled so SSE streams token-by-token.
- Secrets (Anthropic API key, tokens) are injected via Kubernetes Secrets; the SQLite database is persisted through a host-path mount so user accounts and chats survive pod restarts.

Recommendations

- Replace host-path storage for SQLite with a managed PVC (or migrate to a managed Postgres) for true multi-node durability.
- Add liveness probes and CPU/memory requests & limits to every Deployment alongside the existing readiness probe.
- Introduce rate-limiting and stronger input validation on the query endpoints, and move Qdrant to a persistent volume so the vector index is not rebuilt on restart.
- Add a CI/CD pipeline that builds and pushes versioned images and performs rolling updates automatically, plus Prometheus/Grafana observability for latency and token usage.



Introduction

Background and Context

The volume of academic literature grows faster than any researcher can read. Keyword search returns links, not answers, and large language models on their own hallucinate citations. ScholarRAG was built as a cloud-native solution to this problem: it combines a curated corpus of 22,000+ research papers with Retrieval-Augmented Generation so that every answer is grounded in real, retrievable sources. By packaging the retrieval service, generation service, vector database and frontend as independent containers and orchestrating them with Kubernetes, the project shows how a modern AI application can be made reproducible, scalable and highly available.

Problem Statement and Goals

Researchers waste hours locating and cross-reading papers, and general chatbots cannot be trusted to cite accurately. The primary goals of this project were:

- **Grounded Answers:** Provide a single interface where a natural-language question returns a synthesized answer backed by actual papers and, where available, linked GitHub implementations.
- **Retrieval Quality:** Combine HyDE query expansion, bi-encoder retrieval and cross-encoder reranking so the most relevant papers surface first instead of the most keyword-similar ones.
- **DevOps Integration:** Deploy the entire stack — Frontend, Backend, and Vector Database — into a multi-node Kubernetes cluster, bridging the gap between AI development and operations.

Scope and Limitations

- **Scope:** User authentication, a streaming RAG query engine, PDF upload and analysis, persistent chat history, and a fully containerized deployment orchestrated on a kind Kubernetes cluster behind an NGINX Ingress.
- **Security:** Secrets are stored in Kubernetes Secrets, but the current build uses session-token auth suitable for demonstration rather than a hardened production identity provider.
- **Persistence:** SQLite is persisted via a host-path mount and Qdrant currently runs a single replica — adequate for the cluster demo but not for large-scale high availability.
- **Scale:** The system is optimized for a local, multi-node kind environment rather than a managed public-cloud cluster such as EKS or AKS.

Innovation Component

The innovation lies not only in *what* was built (a research assistant) but in *how* it retrieves and how it is shipped:

- **Two-Stage Retrieval:** A fast bi-encoder narrows 22k papers to a candidate set, then a cross-encoder re-scores each candidate against the query for precision — a technique used in production search systems, applied here end-to-end.
- **HyDE Query Expansion:** The system asks the LLM to draft a hypothetical answer and embeds *that* for search, closing the vocabulary gap between short questions and dense abstracts; results are cached for determinism and speed.
- **Streaming, Self-Healing Delivery:** Answers stream token-by-token over SSE through an Ingress tuned to disable buffering, while Kubernetes provides self-healing and rolling updates across a three-node cluster.



Project Objectives

Clearly Defined Objectives and Goals

The primary objective was to engineer a robust, containerized AI application that answers research questions through a grounded, streaming pipeline. Specifically, the project aimed to:

- **Implement a Multi-Tier Architecture:** A decoupled system where the React frontend, FastAPI backend, and Qdrant vector database operate as independent, separately scalable services.
- **Build a High-Quality RAG Pipeline:** Combine HyDE expansion, semantic vector search and cross-encoder reranking with a tunable relevance floor to maximize answer grounding.
- **Master Container Orchestration:** Move beyond localhost by deploying the whole stack into a multi-node Kubernetes (kind) cluster, focusing on service discovery, Ingress routing and rolling updates.
- **Stream Responses in Real Time:** Deliver answers token-by-token over Server-Sent Events so the user sees the model think, with the Ingress configured to pass streams through unbuffered.
- **Secure & Persist State:** Inject API keys via Kubernetes Secrets and persist user accounts and chat history so they survive pod restarts.

Expected Outcomes and Deliverables

Technical Deliverables

- **Source Code:** A React frontend and a FastAPI backend implementing the full RAG pipeline (*rag.py*) and REST API (*main.py*).
- **Ingestion Pipeline:** Scripts that fetch, filter and resample 362k raw records down to a curated 22k-paper corpus (10k Papers-with-Code + 12k OpenAlex) and embed them into Qdrant.
- **Container Images:** Versioned Docker images per service (*scholarrag:backend-v17*, *frontend-v3*, *qdrant-v1*).
- **Orchestration Manifests:** A complete set of Kubernetes YAML under *./k8s* defining Deployments, Services, an NGINX Ingress, a Secret, and the kind cluster configuration.
- **RESTful API:** A structured surface handling auth, streaming queries, PDF upload and chat management.

Expected Outcomes

- **Grounded, Cited Answers:** Responses backed by real papers and, when relevant, links to their GitHub implementations — not unverifiable free text.
- **Deployment Portability:** A “write once, run anywhere” setup whose manifests can be pushed to a managed cloud cluster (EKS/AKS) with minimal change.
- **Resilience:** Two backend and two frontend replicas with rolling updates and a readiness probe so the service stays available during redeloys.



Methodology and Results

Methods & Technology Used

The project followed an Agile methodology, splitting the system into independent components (Frontend, Backend/RAG, Vector Database, Orchestration) so each could be developed, dockerized and tested in isolation.

- **Data Ingestion:** Research papers were fetched from Papers-with-Code and OpenAlex, deduplicated, resampled and embedded with the **all-MiniLM-L6-v2** sentence-transformer into a Qdrant collection.
- **Backend & RAG:** Built with FastAPI. The pipeline runs HyDE expansion → vector search → cross-encoder rerank → Claude generation, with an agentic *fetch_url* tool that reads GitHub READMEs and arXiv pages on demand.
- **Frontend:** A React SPA with authentication, multi-chat history, drag-and-drop PDF upload and live token streaming.
- **Containerization & Orchestration:** Every service is dockerized; Kubernetes (kind) manages deployment, replicas, service discovery and Ingress routing.

Tools & Software Used

Category	Tools
Backend & API	Python, FastAPI, Uvicorn
RAG & AI	Anthropic Claude, HyDE, HuggingFace all-MiniLM-L6-v2 (bi-encoder), cross-encoder ms-marco-MiniLM-L-6-v2, PyMuPDF
Vector Database	Qdrant (“papers” collection)
Frontend	React, JavaScript, Server-Sent Events (SSE)
Data Store	SQLite (users & chat history)
DevOps & Infra	Docker, Kubernetes (kind), kubectl, NGINX Ingress, AWS EC2
Version Control	Git & GitHub

Project Architecture

ScholarRAG follows a cloud-native 3-tier architecture inside the Kubernetes cluster:

- **1. Client Tier (Frontend):** A React SPA served by two frontend pods on port 80. All API traffic is routed to it or the backend by the NGINX Ingress.
- **2. Application Tier (Backend + RAG):** Two FastAPI pods on port 8000 run the RAG pipeline and business logic (auth, retrieval, reranking, streaming generation).
- **3. Data Tier:** A Qdrant pod (port 6333) holds the paper embeddings, and a SQLite database persisted via a host-path mount stores users and chat history.

Traffic flow: User → NGINX Ingress → Frontend (port 80) for the UI, or → */api/** → Backend (port 8000) → Qdrant (port 6333) + Anthropic Claude, with the generated answer streamed back to the browser over SSE.

Final Project Working Results



1. RAG Query & Streaming Response

A user question is expanded via HyDE, embedded and searched against Qdrant; the top candidates are reranked by the cross-encoder, filtered by the relevance floor, and passed to Claude, which streams a grounded answer with paper citations token-by-token to the UI.

2. PDF Upload & Analysis

An uploaded PDF is parsed with PyMuPDF; the abstract is anchored as the retrieval query and a lower PDF relevance floor keeps closely related papers, so the model answers relative to the uploaded document specifically.

3. Kubernetes Deployment & Pod Health

`kubectl get pods` shows backend (x2), frontend (x2) and Qdrant running and Ready across the three-node kind cluster; the readiness probe on **/health** gates traffic and rolling updates redeploy with zero downtime (`maxUnavailable: 0`).

Project GitHub Link

The full source code — ingestion scripts, backend RAG pipeline, React frontend, Dockerfiles and Kubernetes YAML manifests — is available in the project repository on GitHub.



Social / Industry Relevance of the Project

ScholarRAG sits at the intersection of two of the most in-demand areas in industry today: applied Generative AI and cloud-native DevOps. Its relevance spans several fronts:

- **Democratizing Research:** Students and independent researchers — including those in rural and under-resourced settings — gain a free, grounded way to explore 22k+ papers without expensive journal tooling.
- **Trustworthy AI:** By grounding every answer in retrievable sources, the project directly addresses hallucination, the single biggest barrier to adopting LLMs in education and enterprise.
- **Industry-Standard Stack:** FastAPI, Qdrant, Docker, Kubernetes and Ingress-based routing are exactly the technologies used to ship real AI products, making the skills gained immediately employable.
- **Reusable Pattern:** The same retrieval-plus-orchestration blueprint applies to legal, medical and internal-knowledge assistants, so the work generalizes well beyond academic papers.



Learning and Reflection

Team Member Learnings

Role / Focus	Technical Learnings	Management & Soft Skills
RAG & Backend	Designed a two-stage retrieval pipeline (HyDE + bi-encoder + cross-encoder) and tuned relevance floors for quality.	Learned to reason about precision/recall trade-offs and to communicate pipeline behaviour to the frontend team.
DevOps & Infra	Built a multi-node kind cluster, Ingress routing and rolling updates; managed Secrets and persistent state.	Internalized Infrastructure-as-Code — one YAML change can make or break the whole system.
Frontend & Integration	Consumed a streaming SSE API in React and handled async token rendering and PDF upload.	Sharpened collaborative debugging — isolating whether a bug lived in the UI or the backend container.

Reflections on Technology

The leap from running the pipeline on localhost to running it inside a Kubernetes cluster was the team's biggest milestone. We saw first-hand that Docker makes an app portable, but Kubernetes makes it production-ready — self-healing pods, rolling updates and Ingress routing turned a script into a service. Getting SSE streaming to survive the Ingress (by disabling proxy buffering) was a genuine “lightbulb” moment about how real infrastructure shapes application behaviour.

Reflections on Collaboration

Working as a team demanded discipline with Git and with shared configuration. We learned that a single changed service name or environment variable in a Kubernetes manifest can break the backend's connection to Qdrant or the database, so we relied on regular sync-ups and clear ownership of the frontend, backend and infra layers to keep everything integrated.

Challenges Overcome

- **Streaming through the Ingress:** Responses buffered instead of streaming until we disabled proxy buffering and raised the read/send timeouts on the NGINX Ingress.
- **Retrieval relevance:** Early answers pulled in off-topic papers; adding cross-encoder reranking with a tunable RERANK_FLOOR (and a separate PDF floor) fixed grounding.
- **Persisting state in a cluster:** User accounts and chats were lost on restart until we mounted the SQLite database via a host-path volume and injected keys through Kubernetes Secrets.
- **Image and env collisions:** Debugging ImagePullBackOff and a QDRANT_PORT env clash taught us how images are pulled and how env precedence works in Kubernetes.

Individual Experience Summaries

• Akriti Yadav (Team Leader)

“Serving as Team Leader for ScholarRAG was both challenging and rewarding. I assigned responsibilities, tracked progress and kept the team aligned to our deadlines. My technical



contributions included collecting research papers from multiple sources and preparing the dataset for the RAG pipeline, creating and versioning the Docker images, and writing Kubernetes YAML manifests for deployment. Balancing leadership with hands-on DevOps work strengthened my communication, decision-making and project-management skills.”

- **Oum Gupta**

“I worked on the Kubernetes orchestration, the RAG pipeline and the API layer. On the infrastructure side I set up the multi-node kind cluster, wrote the Deployment, Service and Ingress manifests, configured NGINX to route /api traffic to the backend and to stream SSE responses unbuffered, and wired API keys through Kubernetes Secrets with rolling updates for zero-downtime redeployments. On the RAG side I built the retrieval pipeline — HyDE query expansion, bi-encoder search over Qdrant and cross-encoder reranking with a tunable relevance floor — and exposed it through the FastAPI endpoints, including the streaming /query/stream route that the frontend calls. This gave me deep, practical experience connecting an AI pipeline to a production-style API and running it on Kubernetes.”

- **Anurag Singh**

“As a member of the ScholarRAG team I contributed to building and deploying a scalable cloud-based application. My responsibilities included migrating data from JSON to SQLite, configuring NGINX as a reverse proxy, containerizing the application with Docker, and deploying the complete multi-tier application on Kubernetes. I collaborated closely with teammates to troubleshoot deployment issues, improve the architecture and automate the workflow, strengthening my knowledge of Docker, Kubernetes, NGINX, SQLite, AWS and DevOps practices.”

- **Shruti Gupta**

“During the ScholarRAG project I served as a Frontend Team Member, developing the web application’s frontend in React with a strong emphasis on responsive design, usability and clean UI. I integrated the frontend seamlessly with the FastAPI backend, gaining hands-on experience in REST API integration, reusable components and application performance. I also built a strong understanding of Retrieval-Augmented Generation, Large Language Models and context-aware response generation — enhancing my full-stack, problem-solving and collaboration skills.”

- **Saurabh Chandra Shukla**

“I worked as a frontend developer and wrote the YAML manifests used for our Kubernetes deployment. Through this I gained hands-on experience integrating Kubernetes configuration into a real RAG-based system and understanding how the deployment pieces fit together.”



Conclusion and Future Scope

Recap of Objectives and Achievements

The project successfully delivered a grounded, cloud-native AI research assistant. By the end of the development cycle the team achieved the following:

- **Working RAG Pipeline:** A full HyDE → bi-encoder → cross-encoder → Claude pipeline over 22k+ papers, returning citation-backed answers streamed in real time.
- **Successful Orchestration:** A multi-node Kubernetes (kind) cluster running frontend, backend and Qdrant behind an NGINX Ingress, with rolling updates and a readiness probe.
- **Reliable State & Secrets:** Persistent user and chat data and API keys injected securely via Kubernetes Secrets.
- **Team Synergy:** A practical laboratory for mastering Dockerization, Kubernetes manifests, REST API design and modern RAG techniques in a collaborative setting.

Future Scope

1. Retrieval & Model Quality

- Expand the corpus beyond 22k papers and add hybrid (dense + keyword) search for higher recall.
- Add answer-level citation highlighting and confidence scoring so users can see exactly which sentence came from which paper.

2. Advanced Infrastructure

- Move SQLite and Qdrant to managed persistent volumes (or Postgres / Qdrant Cloud) for true high availability, and add Horizontal Pod Autoscaling on the backend.
- Deploy to a managed cloud cluster (AWS EKS / Azure AKS) with TLS termination at the Ingress.

3. Observability & CI/CD

- Integrate Prometheus and Grafana to monitor pod health, query latency and token usage.
- Establish a CI/CD pipeline (GitHub Actions) so every push builds a versioned image and triggers a rolling update automatically.

4. Expanded Features

- Add user-facing citation export (BibTeX) and saved-search alerts for newly ingested papers.
- Support multi-document PDF sessions and follow-up question threads grounded in the uploaded corpus.



Appendix – Deployment Commands & Code Snippets

This appendix lists the key commands used to run ScholarRAG locally and to deploy it to the Kubernetes cluster, along with representative code snippets from the RAG pipeline and API.

A. Local Development Servers

Backend (FastAPI + Uvicorn) — from ScholarRAG/backend

```
# create & activate the virtual environment
python -m venv myenv
myenv\Scripts\activate          # Windows PowerShell
pip install -r requirements.txt

# run the API with hot reload on http://localhost:8000
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Frontend (React + Vite) — from ScholarRAG/frontend

```
npm install
npm run dev          # dev server on http://localhost:5173
npm run build        # production build into dist/
```

B. Docker

Build, tag and push the three service images

```
# backend
docker build -t akritiyadav/schollarrag:backend-v17 ./backend
docker push akritiyadav/schollarrag:backend-v17

# frontend
docker build -t akritiyadav/schollarrag:frontend-v3 ./frontend
docker push akritiyadav/schollarrag:frontend-v3

# run the whole stack locally (frontend on http://localhost:80)
docker compose up -d
docker compose logs -f backend
docker system prune -af          # reclaim disk before a rebuild
```

C. Kubernetes (kind cluster)

Create the cluster and deploy in order (the Ingress rules must be applied after the controller is Ready, or the admission webhook rejects them)

```
# 1. create the 1 control-plane + 2 worker cluster
kind create cluster --name schollarrag --config k8s/kind-config.yaml

# 2. install the NGINX ingress controller, then wait for it
kubectl apply -f k8s/ingress-controller.yaml
kubectl wait --namespace ingress-nginx --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller --timeout=90s

# 3. inject secrets (Anthropic key, tokens) from the .env file
kubectl create secret generic schollarrag-secret --from-env-file=.env

# 4. deploy the three tiers, then the routing rules
kubectl apply -f k8s/qdrant-deployment.yaml
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/frontend-deployment.yaml
kubectl apply -f k8s/ingress.yaml
```



Verify, roll out updates, and tear down

```
kubectl get pods -o wide          # all pods Running / Ready
curl http://localhost:8080/api/health  # {"status":"ok","version":"v17"}

# ship a new backend image with zero downtime (maxUnavailable: 0)
kubectl set image deployment/backend backend=akritiyadav/scholarrag:backend-v17
kubectl rollout status deployment/backend

kind delete cluster --name scholarrag    # full teardown
```

D. Code Snippets

Two-stage retrieval — bi-encoder search then cross-encoder rerank (backend/rag.py)

```
def _search(query_text, k=5):
    vector = _embeddings.embed_query(query_text)    # all-MiniLM-L6-v2
    return _qdrant.query_points(
        collection_name=COLLECTION, query=vector, limit=k).points

def _rerank(query, candidates, top_n):
    pairs = [[query, (r.payload['title'] + '. ' +
                      r.payload['summary'])[:1000]] for r in candidates]
    scores = _cross_encoder.predict(pairs)          # ms-marco-MiniLM
    ranked = sorted(zip(candidates, scores),
                    key=lambda cs: cs[1], reverse=True)
    return [(c, float(s)) for c, s in ranked[:top_n]]
```

HyDE query expansion with caching (backend/rag.py)

```
def _hyde_query(question):
    if question in _hyde_cache:                    # deterministic + fast
        return _hyde_cache[question]
    resp = _client.messages.create(
        model='claude-haiku-4-5-20251001', max_tokens=200,
        messages=[{'role': 'user',
                    'content': f'Write a short passage answering: {question}'}])
    _hyde_cache[question] = resp.content[0].text
    return _hyde_cache[question]
```

Streaming API endpoint over Server-Sent Events (backend/main.py)

```
@app.post('/query/stream')
async def query_stream(question: str = Form(...), model: str = Form(...)):
    async def gen():
        async for token in query_rag_stream(question, session_id, model=model):
            yield f'data: {json.dumps({"token": token})}\n\n'
    return StreamingResponse(gen(), media_type='text/event-stream')
```

Ingress annotations that let SSE stream unbuffered (k8s/ingress.yaml)

```
metadata:
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$1
    nginx.ingress.kubernetes.io/proxy-buffering: "off"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
# route: /api/(.*) -> backend:8000 , /(.*?) -> frontend:80
```